

Experiment- 9

Branch: MCA (AI&ML)	Semester: 2
Student Name: Ajay Rakwal	UID: 25MCI10329
Subject Name: Technical Training	Subject Code: 25CAP-652
Section/Group: MAM-1(A)	Date of Performance: 07-04-2026

Aim of the program:

To implement database triggers in PostgreSQL to automatically calculate values and enforce constraints during data insertion operations.

Software Requirements

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench / Web Based / pgAdmin

Objective

- To understand the concept and working of database triggers in PostgreSQL.
- To learn how to automatically calculate derived values during data insertion.
- To implement BEFORE INSERT triggers for enforcing business rules.
- To apply constraints using trigger logic to validate data before insertion.
- To understand how triggers help maintain data integrity and automation in databases

Procedure of the Practical

- Open PostgreSQL (pgAdmin) and connect to the required database.
- Create the Employee table with fields such as emp_id, emp_name, working_hours, perhour_salary, and total_payable_amount.
- Write a trigger function using PL/pgSQL to calculate the total payable amount based on working hours and per-hour salary.
- Add validation logic inside the function to restrict insertion if the total payable amount exceeds the specified limit.
- Create a BEFORE INSERT trigger and associate it with the Employee table to execute the function automatically.
- Insert sample records into the Employee table to test the trigger functionality.
- Observe the behavior for valid and invalid inputs (accepted and rejected records).
- Retrieve and verify the inserted records using a SELECT query.

Practical / Experiment Steps

-- STEP 1: CREATE EMPLOYEE TABLE

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    working_hours INT,  
    perhour_salary NUMERIC,  
    total_payable_amount NUMERIC);
```

-- STEP 2: CREATE TRIGGER FUNCTION TO CALCULATE PAYABLE AMOUNT

```
CREATE OR REPLACE FUNCTION CACULATE_PAYABLE_AMOUNT() RETURNS  
TRIGGER
```

```
AS
```

```
$$
```

BEGIN

-- CALCULATE TOTAL PAYABLE AMOUNT

NEW.total_payable_amount := NEW.perhour_salary * NEW.working_hours;

-- STEP 3: APPLY VALIDATION CONDITION (MAX LIMIT CHECK)

IF NEW.total_payable_amount > 25000 THEN

RAISE EXCEPTION 'INVALID ENTRY BEACUSE PAYABLE AMOUNT CAN NOT
BE GREATER THAN 25000';

END IF;

-- RETURN UPDATED RECORD

RETURN NEW;

END;

\$\$ LANGUAGE PLPGSQL;

-- STEP 4: CREATE TRIGGER TO EXECUTE FUNCTION BEFORE INSERT

CREATE OR REPLACE TRIGGER

AUTOMATED_PAYABLE_AMOUNT_CALCULATION

BEFORE INSERT

ON employee

FOR EACH ROW

EXECUTE FUNCTION CACULATE_PAYABLE_AMOUNT();

-- STEP 5: INSERT VALID RECORD (SHOULD BE ACCEPTED)

INSERT INTO EMPLOYEE (EMP_ID, EMP_NAME, working_hours, perhour_salary)

VALUES (1, 'Ajay', 15, 1000);

-- STEP 6: INSERT INVALID RECORD (SHOULD THROW ERROR)

INSERT INTO EMPLOYEE (EMP_ID, EMP_NAME, working_hours, perhour_salary)

VALUES (2, 'Purnima', 8, 100000);

-- STEP 7: DISPLAY FINAL TABLE DATA

SELECT * FROM EMPLOYEE;

I/O Analysis (Input / Output)

Input

- Table creation query for Employee
- Trigger function definition for calculating payable amount
- Trigger creation statement (BEFORE INSERT trigger)
- INSERT queries with employee details (working hours and per-hour salary)

Output

- Employee table created successfully
- Total payable amount automatically calculated during insertion
- Valid records inserted correctly into the table
- Invalid records (payable amount > 25000) rejected with an error message
- Data integrity maintained through trigger-based validation

OUTPUT:

--Table Created:

```
Data Output Messages Notifications
CREATE TABLE

Query returned successfully in 70 msec.
```

--CREATE TRIGGER FUNCTION AND APPLY VALIDATION:

```
Data Output Messages Notifications
CREATE FUNCTION

Query returned successfully in 588 msec.
```

-- INSERT VALID RECORD (SHOULD BE ACCEPTED)

```
Data Output Messages Notifications
INSERT 0 1

Query returned successfully in 128 msec.
```

-- INSERT INVALID RECORD (SHOULD THROW ERROR)

```
Data Output Messages Notifications
ERROR:  INVALID ENTRY BEACUSE PAYABLE AMOUNT CAN NOT BE GREATER THAN 25000
CONTEXT:  PL/pgSQL function caculate_payable_amount() line 8 at RAISE

SQL state: P0001
```

-- DISPLAY FINAL TABLE DATA

Data Output	Messages	Notifications
emp_id [PK] integer	emp_name character varying (50)	working_hours integer
perhour_salary numeric	total_payable_amount numeric	
1	1 Ajay	15 1000 15000

Learning Outcome:

- Understand how triggers automate database operations without manual intervention.
- Learn to implement trigger functions using PL/pgSQL.
- Gain practical experience in applying business rules and constraints using triggers.
- Understand the role of triggers in maintaining data consistency and validation.
- Develop the ability to design automated database systems for real-world applications.